

Hanoi als microprobleem: patroon, recursie en semantisch begrip

J.W. Krol

Juli 2026

Inhoudsopgave

1 Inleiding	1
2 Hanoi als stack-flow-probleem	2
3 De recursieve oplossing	3
4 De iteratieve binaire oplossing	5
5 Twee representaties van hetzelfde proces	7
6 Patroonherkenning en begrip	7
7 Mens, machine en semantische modellen	8
8 Beschouwing: patroon en AGI	9
9 Conclusie	9
10 Slotconclusie: programmeren als abstract model en fysische uitvoering	10
A Hanoi met recursie	11

1 Inleiding

De torens van Hanoi lijken op het eerste gezicht een eenvoudig recreatief probleem. Men heeft drie pennen of stacks en een aantal schijven van verschillende grootte. De opdracht is om alle schijven van de eerste stack naar de derde stack te verplaatsen, waarbij nooit een grotere schijf op een kleinere mag worden geplaatst.

Juist door deze eenvoud is het probleem interessant. In een klein en volledig overzienbaar domein worden meerdere fundamentele lagen van algoritmisch denken zichtbaar: toestand, transformatie, invariant, recursie, patroonvorming en machine-uitvoering.

De centrale observatie in dit artikel is dat Hanoi niet primair hoeft te worden gezien als een probleem over schijven, maar als een probleem over stack-flow. De elementaire operatie is steeds dezelfde:

$\text{push}(Y, \text{pop}(X)).$

Een zet wordt daardoor volledig bepaald door de bronstack X en de doelstack Y . De schijf zelf hoeft niet in de instructie te worden genoemd; zij wordt bepaald door de toestand van de bronstack.

Daarmee wordt Hanoi een formeel proces:

toestand + primitieve operatie + move-string.

De vraag wordt dan: welk woord over het alfabet van stacks transformeert de begintoestand naar de doeltoestand onder behoud van de invariant?

2 Hanoi als stack-flow-probleem

Neem drie stacks:

$A, \quad B, \quad C.$

De begintoestand is:

$A = [1, 2, \dots, n], \quad B = [], \quad C = [],$

waarbij 1 de kleinste schijf is en bovenaan ligt. De doeltoestand is:

$A = [], \quad B = [], \quad C = [1, 2, \dots, n].$

De enige primitieve operatie is:

$XY \equiv \text{push}(Y, \text{pop}(X)).$

Hierbij betekent XY dus: verplaats de bovenste schijf van stack X naar stack Y .

De invariant is:

elke stack blijft geordend van klein naar groot.

Met andere woorden: een move XY is alleen geldig als X niet leeg is en als Y leeg is of de bovenste schijf van X kleiner is dan de bovenste schijf van Y .

In pseudocode:

```
allowed(X, Y):  
    if empty(X):  
        return false  
  
    if empty(Y):
```

```
return true

return top(X) < top(Y)
```

De feitelijke move is:

```
move(X, Y):
    push(Y, pop(X))
```

De abstracte vorm van het probleem wordt daarmee:

```
gegeven:
    stacks A, B, C
    primitive move XY = push(Y, pop(X))
    invariant: elke stack blijft geordend
    goal: alle schijven van A naar C

vind:
    een geldige move-string van minimale lengte
```

Voor $n = 3$ is de minimale oplossing:

$AC, AB, CB, AC, BA, BC, AC.$

Als compacte string:

$ACABCBACBABCAC,$

Het algoritme is dus een woord over het alfabet

$\{A, B, C\}.$

De betekenis van het woord ontstaat pas wanneer het wordt geïnterpreteerd door de stack-machine die de primitieve operatie XY uitvoert.

3 De recursieve oplossing

De klassieke oplossing van Hanoi is recursief. Om n schijven van A naar C te verplaatsen met B als hulpstack, doet men:

1. verplaats $n - 1$ schijven van A naar B ,
2. verplaats de grootste schijf van A naar C ,
3. verplaats $n - 1$ schijven van B naar C .

Formeel:

$$H(n, A, B, C) = H(n - 1, A, C, B) \text{ AC } H(n - 1, B, A, C).$$

Het basisgeval is:

$$H(1, A, B, C) = AC.$$

In pseudocode:

```
hanoi(n, A, B, C):  
    if n == 1:  
        move(A, C)  
    else:  
        hanoi(n-1, A, C, B)  
        move(A, C)  
        hanoi(n-1, B, A, C)
```

Voor $n = 3$ geeft dit:

```
hanoi(3, A, B, C)  
    hanoi(2, A, C, B)  
        hanoi(1, A, B, C) -> AC  
        move(A, B)        -> AB  
        hanoi(1, C, A, B) -> CB  
    move(A, C)             -> AC  
    hanoi(2, B, A, C)  
        hanoi(1, B, C, A) -> BA  
        move(B, C)        -> BC  
        hanoi(1, A, B, C) -> AC
```

Dus:

$$H(3, A, B, C) = AC, AB, CB, AC, BA, BC, AC.$$

De lengte $T(n)$ van de move-string voldoet aan:

$$T(n) = 2T(n-1) + 1, \quad T(1) = 1.$$

Daaruit volgt:

$$T(n) = 2^n - 1.$$

Deze formule is niet alleen een telling van de recursieve methode; zij is ook minimaal. Om de grootste schijf te kunnen verplaatsen, moeten eerst alle $n - 1$ kleinere schijven van de bronstack af. Daarna moet de grootste schijf worden verplaatst. Vervolgens moeten de $n - 1$ kleinere schijven weer bovenop de grootste worden geplaatst. Daardoor is minimaal nodig:

$$T(n-1) + 1 + T(n-1).$$

De recursieve oplossing is dus niet alleen correct, maar ook optimaal.

4 De iteratieve binaire oplossing

De recursieve oplossing volgt direct uit de semantische structuur van het probleem. Er bestaat echter ook een iteratieve methode die dezelfde minimale move-string genereert zonder expliciete recursie.

Kijk opnieuw naar de oplossing voor $n = 3$:

$$AC, AB, CB, AC, BA, BC, AC.$$

De bijbehorende schijven die bewegen zijn:

$$1, 2, 1, 3, 1, 2, 1.$$

Dit patroon wordt bepaald door de binaire structuur van het zetnummer. Voor zetnummer t geldt:

$$d(t) = 1 + \text{ctz}(t),$$

waarbij $\text{ctz}(t)$ het aantal trailing zero bits is in de binaire representatie van t . Anders gezegd: $\text{ctz}(t)$ is het aantal keren dat t door 2 deelbaar is.

Voorbeelden:

t	binair	$\text{ctz}(t)$	$d(t)$
1	001	0	1
2	010	1	2
3	011	0	1
4	100	2	3
5	101	0	1
6	110	1	2
7	111	0	1

Dus voor $n = 3$:

$$d(1), d(2), d(3), d(4), d(5), d(6), d(7) = 1, 2, 1, 3, 1, 2, 1.$$

In C-termen zou men schrijven:

```
disk = ctz(t) + 1;
```

Daarmee is bekend welke schijf moet bewegen. Vervolgens moet men weten in welke richting deze schijf cyclisch over de drie stacks beweegt.

Elke schijf heeft een vaste cyclische richting:

$$A \rightarrow B \rightarrow C \rightarrow A$$

of

$$A \rightarrow C \rightarrow B \rightarrow A.$$

Welke richting geldt, hangt af van de pariteit van n en van de schijf. Voor het klassieke doel $A \rightarrow C$ geldt:

- als n oneven is, beweegt de kleinste schijf volgens

$$A \rightarrow C \rightarrow B \rightarrow A;$$

- als n even is, beweegt de kleinste schijf volgens

$$A \rightarrow B \rightarrow C \rightarrow A.$$

De richtingen van opeenvolgende schijven wisselen. Men kan dit uitdrukken door aan elke schijf d een richting toe te kennen afhankelijk van de pariteit van $n - d$. Concreet betekent dit dat wanneer schijf 1 met de klok mee over de stacks beweegt, schijf 2 de andere kant op draait, schijf 3 weer met de klok mee gaat, enzovoort.

De iteratieve generator heeft dan de vorm:

```
for t = 1 to 2^n - 1:
  d = ctz(t) + 1
  verplaats schijf d een stap in zijn vaste cyclische richting
```

Men hoeft hiervoor geen recursieve call stack te gebruiken. Het geheugen dat nodig is, bestaat uit:

```
t      : huidig zetnummer
n      : aantal schijven
pos[d] : huidige stackpositie van schijf d
dir[d] : cyclische richting van schijf d
```

Voor $n = 3$ ontstaat:

```
t=1: d=1: A -> C = AC
t=2: d=2: A -> B = AB
t=3: d=1: C -> B = CB
t=4: d=3: A -> C = AC
t=5: d=1: B -> A = BA
t=6: d=2: B -> C = BC
t=7: d=1: A -> C = AC
```

Dit is exact dezelfde move-string als bij de recursieve oplossing:

$$AC, AB, CB, AC, BA, BC, AC.$$

5 Twee representaties van hetzelfde proces

De recursieve en de iteratieve oplossing produceren dezelfde minimale flow, maar zij representeren het probleem op een totaal verschillende manier.

De recursieve oplossing zegt:

los n op door tweemaal $n - 1$ op te lossen.

Zij is semantisch. Zij maakt duidelijk waarom het probleem oplosbaar is, waarom de invariant behouden blijft en waarom de oplossing generaliseert naar elk n .

De iteratieve oplossing zegt:

gebruik het zetnummer om direct de actieve schijf te bepalen.

Zij is patroonmatig. Zij gebruikt de verborgen binaire regelmaat in de gegenereerde move-string.

Men kan het verschil als volgt samenvatten:

Recursief	Iteratief binair
deelprobleemstructuur	stapnummerstructuur
volledige inductie	trailing-zero-regel
semantisch begrip	patrooncompressie
call stack	teller en posities
waarom het werkt	hoe men de volgende zet berekent

Beide zijn correct. Beide zijn algoritmisch. Maar zij drukken een ander soort inzicht uit.

6 Patroonherkenning en begrip

Hanoi laat zien dat correct gedrag niet hetzelfde is als begrip. Dezelfde reeks zetten kan op verschillende manieren ontstaan:

1. door memorisatie van de move-string,
2. door een binaire patroonregel,
3. door recursief inzicht in de structuur van het probleem.

Voor $n = 3$ kan een systeem eenvoudig de reeks leren:

$AC, AB, CB, AC, BA, BC, AC.$

Een sterker systeem kan de binaire regel ontdekken:

$$d(t) = 1 + \text{ctz}(t).$$

Maar het semantische begrip bestaat uit iets anders. Het bestaat uit het herkennen van:

toestand:
drie geordende stacks
operatie:
$XY = \text{push}(Y, \text{pop}(X))$
constraint:
never a larger disk on a smaller one
invariant:
elke stack blijft geordend
doel:
verplaats alle schijven van A naar C
genererende regel:
$H(n, A, B, C) =$
$H(n-1, A, C, B)$
AC
$H(n-1, B, A, C)$

Dit model verklaart waarom de oplossing werkt. Het verklaart ook waarom zij generaliseert. Dat is het verschil tussen correlatie en theorie.

Een patroonmodel kan leren:

spoor $\rightarrow$ waarschijnlijk volgend spoor.

Een semantisch model probeert daarentegen:

spoor $\rightarrow$ procesmodel $\rightarrow$ nieuwe geldige sporen.

De eerste vorm is patroonherkenning. De tweede vorm is begrip.

7 Mens, machine en semantische modellen

Een machine kan de iteratieve binaire oplossing zeer efficiënt uitvoeren. Zij hoeft niet te begrijpen waarom Hanoi werkt; zij hoeft alleen de teller t , de functie ctz , de posities van de schijven en de richtingen te gebruiken.

Het menselijk brein vindt vaak de recursieve oplossing natuurlijker, omdat zij aansluit bij betekenisvolle decompositie:

groot probleem $\rightarrow$ kleiner probleem $\rightarrow$ basisgeval.

Dat is niet primair een uitvoeringsmodel, maar een begripsmodel. De recursieve beschrijving toont de structuur van het probleem. Zij maakt de invariant, de reductie en de noodzaak van de stappen zichtbaar.

Daarmee ontstaat een belangrijk onderscheid:

Machine-uitvoering	Menselijk begrip
instructiestroom	betekenisvolle structuur
teller en geheugen	toestand en doel
patroongenerator	semantisch model
correcte actie	verklaring van correctheid

Programmeertalen vormen de brug tussen deze twee werelden. Zij maken het mogelijk om een menselijke probleemstructuur te vertalen naar een uitvoerbaar proces op een machine.

8 Beschouwing: patroon en AGI

De moderne machine-learningbenadering is zeer succesvol in het herkennen en genereren van patronen. In veel domeinen is dat voldoende om indrukwekkend gedrag te produceren. Maar Hanoi laat in een klein gesloten domein zien dat patroonherkenning niet noodzakelijk hetzelfde is als semantisch begrip.

Een systeem kan de juiste zetten produceren zonder het probleem werkelijk te begrijpen. Het kan de move-string memoriseren. Het kan een statistische regelmaat leren. Het kan zelfs de binaire patroonregel vinden. Maar pas wanneer het systeem de toestand, de operatie, de constraint, de invariant en de genererende regel reconstrueert, ontstaat iets dat op begrip lijkt.

Voor algemene intelligentie is dit onderscheid fundamenteel. Een systeem dat alleen patronen leert, blijft kwetsbaar voor situaties waarin de zichtbare correlaties veranderen. Een systeem met een semantisch model kan daarentegen redeneren over waarom een patroon bestaat, waar het geldig is en hoe het naar nieuwe situaties kan worden overgedragen.

Men kan dit samenvatten als:

patroonherkenning = regelmaat in sporen vinden,

terwijl:

begrip = het proces reconstrueren dat de sporen voortbrengt.

In deze zin is Hanoi meer dan een puzzel. Het is een microkosmos van het verschil tussen patroon en theorie, tussen uitvoer en verklaring, tussen machinegedrag und menselijk begrip.

9 Conclusie

Door Hanoi te formuleren als stack-flow-probleem wordt de essentie van het probleem zichtbaar. De primitieve operatie is slechts:

$$XY = \text{push}(Y, \text{pop}(X)).$$

De oplossing is een move-string over de stacknamen A, B, C . De recursieve methode genereert deze string vanuit volledige inductie en semantische decompositie. De iteratieve binaire methode genereert dezelfde string vanuit een patroonregel op het zetnummer.

Daarmee wordt duidelijk dat dezelfde uitvoer verschillende oorsprongen kan hebben: memorisatie, patrooncompressie of semantisch begrip. Dit kleine probleem laat daardoor zien waarom correcte output niet voldoende is als criterium voor begrip. Begrip vereist een model van toestand, transformatie, invariant, doel en genererende structuur.

Hanoi toont dus in miniatuur een algemene les over algoritmen, menselijke cognitie en kunstmatige intelligentie: patronen zijn sporen van processen, maar begrip ontstaat pas wanneer het proces achter het patroon wordt gereconstrueerd.

10 Slotconclusie: programmeren als abstract model en fysische uitvoering

Dit onderscheid sluit aan bij een centraal thema uit *Structure and Interpretation of Computer Programs* [5]. Ook daar wordt programmeren niet slechts voorgesteld als het schrijven van instructies, maar als het construeren van abstracte processen. Tegelijk laat SICP zien dat zulke abstracte processen uiteindelijk moeten worden geïnterpreteerd of uitgevoerd door een concreet evaluatiemodel: een omgeving, een evaluator, een expliciete-control evaluator of een registermachine.

Daarmee wordt zichtbaar dat een programma twee gezichten heeft. Aan de ene kant is het een abstracte beschrijving van een proces; aan de andere kant moet het als een mechanische uitvoering gerealiseerd kunnen worden. De hogere orde van programmeren ligt in het construeren van het abstracte model, maar de betekenis daarvan wordt pas fysisch werkzaam wanneer het model wordt uitgevoerd door een machine.

De analyse van Hanoi laat in miniatuur zien wat programmeren in algemene zin is. Programmeren is niet slechts het schrijven van instructies voor een machine, maar het ontwerpen van een abstract model van een proces. Dat model beschrijft toestanden, toegestane transformaties, invarianten en doelen.

Om werkelijk uitgevoerd te worden, moet dit abstracte model echter worden vertaald naar een mechanistisch proces. Op een computer betekent dit dat het model wordt omgezet in een reeks elementaire operaties op geheugen, registers, stacks en instructiestromen. De abstracte structuur wordt daardoor fysisch realiseerbaar.

Tijdens die uitvoering ontstaat een spoor: een meetbare opeenvolging van toestanden, geheugenveranderingen, stackoperaties en instructies. Dit spoor vormt een patroon. Maar dat patroon is niet de betekenis zelf; het is de fysische afdruk van het abstracte model dat wordt uitgevoerd.

Bij Hanoi is dit direct zichtbaar. Het abstracte model bestaat uit drie geordende stacks, de operatie

$$XY = \text{push}(Y, \text{pop}(X)),$$

de invariant dat elke stack geordend blijft, en het doel om de inhoud van A naar C te verplaatsen. De uitvoering laat vervolgens een move-string achter, bijvoorbeeld:

$$AC, AB, CB, AC, BA, BC, AC.$$

Deze string is het spoor van de uitvoering. Zij weerspiegelt de betekenis van het abstracte model, maar zij verklaart die betekenis niet op zichzelf. Zonder kennis van stacks, operatie, invariant en doel is de string slechts een patroon. Met dat semantische model wordt zij begrijpelijk als noodzakelijke transformatie.

Daarmee ontstaat de algemene conclusie:

$$\text{abstract model} \rightarrow \text{mechanistische uitvoering} \rightarrow \text{fysisch spoor} \rightarrow \text{patroon}.$$

Programmeren is dus het maken van abstracte modellen die op een mechanische wijze fysisch kunnen worden uitgevoerd. De mechanistische uitvoering laat een spoor achter, en dat spoor verschijnt als patroon. Maar het patroon krijgt pas betekenis vanuit het abstracte model dat het heeft voortgebracht.

Dit onderscheid is ook essentieel voor het begrijpen van kunstmatige intelligentie. Een systeem dat alleen het spoor leert, leert een patroon. Een systeem dat het model achter het spoor reconstrueert, nadert begrip. AGI is niet primair het herkennen van patronen, maar het vormen van abstracte procesmodellen die patronen kunnen voortbrengen, verklaren en mechanistisch uitvoeren.

A Hanoi met recursie

In deze sectie ontwikkelen we de eenvoudige implementatie van Hanoi met de recursie methode in LUA. De code van deze appendix kan hier gevonden worden: [Hanoi](#)

We beginnen met de implementatie van een stack. LUA kent als datastructuur de tabel. We kunnen een klasse van objecten maken met methoden via de metatable die aan een table kan worden toegevoegd.

```
1 local Stack = {}  
2 Stack.__index = Stack
```

We definiëren een constructor voor een Stack met een property name.

```
1 function Stack:new(naam)  
2     local instance = {  
3         name = naam or "Onbekende Stack" -- Sla de naam op  
4     }  
5     setmetatable(instance, Stack)  
6     return instance  
7 end
```

Elementen worden oplopend met een index 1..n toegevoegd. We kunnen dus de push, pop en seek methode eenvoudig toevoegen.

```

1  -- voeg een waarde aan de top toe
2  function Stack:push(waarde)
3      table.insert(self, waarde)
4  end
5
6  -- geef top element en verwijder, nil voor een lege stack
7  function Stack:pop()
8      return table.remove(self)
9  end
10
11 -- geef top element maar verwijder niet
12 function Stack:peek()
13     return self[#self] --#self aantal elementen van de stack
14 end
15
16 function Stack:is_empty()
17     return #self == 0
18 end

```

Tot slot een functie om de stack te printen.

```

1  function Stack:__tostring()
2      local elementen = {}
3
4      -- Loop door alle genummerde indexen (de inhoud van de stack)
5      for i = 1, #self do
6          -- we zetten tostring() eromheen voor het geval er getallen/booleans in zitten
7          table.insert(elementen, tostring(self[i]))
8      end
9
10     -- Plak alle elementen aan elkaar met een komma en spatie
11     local inhoud_tekst = table.concat(elementen, ", ")
12
13     -- Geef de geformatteerde string terug
14     return "Stack: " .. self.name .. " [" .. inhoud_tekst .. "]"
15 end

```

Om de Stack module in andere programma's te kunnen gebruiken moeten we de tabel retourneren:

```

1  return Stack

```

Vervolgens kunnen we het recursief algoritme met deze stacks maken. We moeten de module Stack importeren en omdat we maken van Hanoi zelf ook een module zodat we de functies kunnen toepassen in test programma's. We voegen een DEBUG mode toe om te checken dat ons algoritme aan de invariant dat een move altijd een kleinere schijf op een grotere schijf plaatst.

```

1 local Stack = require("stack")
2
3 local Hanoi = {}
4
5 -- debug mode
6 Hanoi.DEBUG = true
7 -- De invariant-check functie
8 local function check_invariant(bron, doel)
9     if Hanoi.DEBUG then
10         -- We checken alleen als de doelstack NIET leeg is
11         if not doel:is_empty() then
12             local schijf_te_verplaatsen = bron:peek()
13             local bovenste_schijf_doel = doel:peek()
14
15             -- Invariant: de schijf die we pakken MOET kleiner zijn dan de top van de
doelstack
16             assert(schijf_te_verplaatsen < bovenste_schijf_doel,
17                 string.format("[DEBUG ERROR] Invariant geschonden! Schijf %d mag niet
op Schijf %d van Stack %s",
18                     schijf_te_verplaatsen, bovenste_schijf_doel, doel.name))
19         end
20     end
21 end
22
23 local function move(x,y)
24     -- debug check
25     check_invariant(x, y)
26
27     y:push(x:pop())
28     io.write(x.name)
29     io.write(y.name)
30     io.write(" ")
31 end
32
33 function Hanoi.recursief(n,from,to,spare)
34     if n==1 then -- stop conditie
35         move(from,to)
36     else
37         Hanoi.recursief(n-1,from,spare,to)
38         move(from,to)
39         Hanoi.recursief(n-1,spare,to,from)
40     end
41 end

```

Voor de iteratieve versie bestaat er een relatie tussen het zetnummer en de schijf nummer: schijf nummer = aantal binaire nullen voor eerste 1 + 1. De functie ctz bepaalt het aantal binaire nullen voor eerste 1 voor een zetnummer.

```

1 local function ctz(t)
2     if t == 0 then return 32 end
3     local count = 0
4     -- Zolang het meest rechtse bitje 0 is, schuiven we op

```

```

5   while (t & 1) == 0 do
6       count = count + 1
7       t = t >> 1
8   end
9   return count
10 end

```

De iteratieve versie werkt met een for loop van 1 tot $2^n - 1$. Afhankelijk van of n even of oneven is zijn er twee pariteiten van de volgorde om de stacks te doorlopen voor een schijf:

- oneven: from, to, spare
- even: from spare, to

```

1  -- iteratieve versie
2  function Hanoi.iteratief(n, from, to, spare)
3
4  local totaal_zetten = 2^n - 1
5
6  local stacks
7      if n % 2 == 1 then
8          stacks = {from, to, spare}
9      else
10         stacks = {from, spare, to}
11     end

```

We moeten per schijf de stack positie bijhouden, initieel bevinden alle schijven zich in from met index = 1. De kleinste schijf (1) draait met de klok mee, en volgende schijven draaien in tegengestelde richting van de voorganger.

```

1  local schijf_posities = {}
2      for d = 1, n do
3          schijf_posities[d] = 1
4      end

```

De iteratieve loop bepaalt de schijf op basis van trailing zeros in het binaire nummer en de huidige schijfpositie. Daaruit kan dan de bron en de doel stack worden afgeleid en de operatie move worden uitgevoerd en de huidige schijfpositie worden bijgewerkt.

```

1  for t = 1, totaal_zetten do
2      -- Bepaal welke schijf moet bewegen via de trailing zeros
3      local d = ctz(t) + 1
4
5      local huidige_stack_idx = schijf_posities[d]
6      local volgende_stack_idx
7
8      -- De oneven schijven draaien met klok mee, de even schijven tegen de klok in
9      if d % 2 == 1 then
10         -- Met de klok mee
11         volgende_stack_idx = (huidige_stack_idx % 3) + 1
12     else
13         -- Tegen de klok in
14         volgende_stack_idx = huidige_stack_idx - 1
15         if volgende_stack_idx == 0 then volgende_stack_idx = 3 end
16     end
17
18     -- Pak de daadwerkelijke Stack-objecten erbij
19     local bron = stacks[huidige_stack_idx]
20     local doel = stacks[volgende_stack_idx]
21
22     -- Voer de fysieke move uit op de stacks
23     move(bron, doel)
24
25     -- Update de positie van deze schijf voor de volgende rondes
26     schijf_posities[d] = volgende_stack_idx
27 end
28 end
29
30 -- return module
31 return Hanoi

```

Hier een voorbeeld voor het gebruik:

```
1 -- Laad de benodigde modules in
2 local Stack = require("stack")
3 local Hanoi = require("hanoi")
4
5 -- Een hulpfunctie om de pennen te resetten naar de beginstand
6 local function maak_beginstand(n)
7     local A = Stack:new("A")
8     local B = Stack:new("B")
9     local C = Stack:new("C")
10
11     -- Vul pen A in omgekeerde volgorde: van n naar 1 (stapgrootte -1)
12     for i = n, 1, -1 do
13         A:push(i)
14     end
15     return A, B, C
16 end
17
18 print("=== TEST 1: RECURSIEVE METHODE ===")
19 local A1, B1, C1 = maak_beginstand(3)
20 Hanoi.recursief(3, A1, C1, B1) -- Roept de recursieve versie aan
21 print("\nEindstand C:", C1)
22 print()
23
24 print("=== TEST 2: ITERATIEVE METHODE ===")
25 local A2, B2, C2 = maak_beginstand(3)
26 Hanoi.recursief(3, A2, C2, B2) -- Roept de binaire versie aan
27 print("\nEindstand C:", C2)
```

Referenties

- [1] Kotovsky, K., Hayes, J. R., & Simon, H. A. (1985). *Why are some problems hard? Evidence from Tower of Hanoi*. *Cognitive Psychology*, 17(2), 248–294.
- [2] Simon, H. A. (1975). *The functional equivalence of problem solving skills*. *Cognitive Psychology*, 7(2), 268–288.
- [3] Gardner, M. (1959). *The Scientific American Book of Mathematical Puzzles and Diversions*. Simon and Schuster. (Hoofdstuk over de binaire connectie en de pariteitsregels van Hanoi).
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (Voor de formele analyse van divide-and-conquer en recursieve complexiteit).
- [5] Abelson, H., Sussman, G. J., & Sussman, J. (1996). *Structure and Interpretation of Computer Programs* (2nd ed.). MIT Press. (Voor het onderscheid tussen programma's als abstracte processen en hun uitvoering via evaluatiemodellen, omgevingen en registermachines).